

Hands-On Graph Neural Networks Using Python

Chapter 7 상세 정리

A7610 그래프신경망특론

2026년 4월 15일

Contents

1	Chapter 7 개요	2
2	GAT의 수학적 직관	2
3	노트북 1부: toy 예제 해설	3
4	노트북 2부: Cora/CiteSeer 실험	4
5	결과 해석	4
6	Degree별 성능 분석	5
7	발제 포인트	5
8	Chapter 7 핵심 요약	6

1 Chapter 7 개요

Chapter 7의 핵심은 **Graph Attention Networks (GAT)**이다. 기존 메시지 패싱 기반 GNN과 달리, GAT는 이웃마다 다른 중요도 계수를 부여해 집계한다.

”같은 이웃 집계라도 모든 이웃을 똑같이 보지 않고, 더 중요한 이웃에 더 큰 가중치를 준다.”

실습 노트북 `practice/hands_on_gnn_examples/Chapter07/chapter7.ipynb` 흐름은 다음과 같다.

1. toy 예제로 attention 계산 절차를 수치적으로 확인
2. PyTorch Geometric으로 GATv2 모델 구현
3. Cora/CiteSeer 데이터셋에서 분류 성능 비교
4. 노드 degree별 정확도 분석으로 성능 편차 해석

노트북의 의도

`chapter7_annotated.ipynb`는 원본 실습 코드에 설명을 삽입한 버전이다.

- 코드 셀은 그대로 실행 가능하게 유지한다.
- 각 코드 블록 앞에 “왜 이 계산이 필요한가”를 설명한다.
- 교재의 핵심 그림 6개를 함께 배치해 수식-코드-도식을 연결한다.

먼저 정리할 오해: GAN이 아니라 GAT

발표 초반에는 용어 혼동을 먼저 끊는 편이 좋다.

- **GAN**: 생성자와 판별자가 경쟁하는 생성 모델
- **GAT**: 그래프 이웃의 중요도를 attention으로 학습하는 메시지 패싱 모델

즉, 이 장의 관심사는 “가짜 샘플 생성”이 아니라 “어떤 이웃의 메시지를 더 크게 들을 것인가”이다.

2 GAT의 수학적 직관

노드 i 의 새로운 표현은 이웃 메시지의 가중합으로 표현된다.

$$\mathbf{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W} \mathbf{h}_j \right)$$

여기서 attention 계수는 다음과 같이 계산된다.

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W} \mathbf{h}_i \parallel \mathbf{W} \mathbf{h}_j]), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}$$

핵심은 **중심 노드별 softmax 정규화**다. 즉, 노드 i 의 이웃들 내부에서 상대적으로 중요한 이웃이 높은 계수를 받는다.

3 노트북 1부: toy 예제 해설

노트북은 4개 노드 인접행렬 A 를 직접 정의하고, 랜덤 초기화된 특징행렬 X , 투영행렬 W , attention 벡터 W_{att} 를 사용해 계수 계산을 시뮬레이션한다.

```
connections = np.where(A > 0)

a = W_att @ np.concatenate([
    (X @ W.T)[connections[0]],
    (X @ W.T)[connections[1]]
], axis=1).T

e = leaky_relu(a)
E = np.zeros(A.shape)
E[connections[0], connections[1]] = e[0]
W_alpha = softmax2D(E, 1)
```

이 구현은 논문 수식을 코드로 어떻게 번역하는지 보여준다.

- 연결된 노드쌍만 점수를 계산한다.
- LeakyReLU 후 각 중심 노드에서 softmax를 적용한다.
- 최종적으로 가중치 행렬 W_α 를 얻는다.

toy example를 읽는 순서

노트북의 설명을 발표용 언어로 옮기면 다음 순서가 가장 명확하다.

1. 입력 준비: A 는 연결 구조, X 는 노드 특징, W 는 특징 변환, W_{att} 는 attention 점수 생성 파라미터다.
2. 연결된 노드쌍 추출: `connections = np.where(A > 0)`로 edge가 있는 쌍 (i, j) 만 남긴다.
3. 원시 점수 계산: `np.concatenate(...)`가 $[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]$ 에 대응한다.
4. 정규화: `softmax2D(E, 1)`에서 축 1은 “한 중심 노드의 이웃 집합”을 뜻한다.
5. 가중합: attention이 큰 이웃일수록 최종 표현 H 에 더 크게 반영된다.

이 과정을 손으로 따라가면 뒤에서 GATv2Conv가 수행하는 일이 블랙박스 아니라는 점이 중요하다.

교재 그림과 코드 연결

교재 그림 7.1–7.2는 노트북 코드와 거의 일대일로 대응된다.

- `connections = np.where(A > 0)`: 실제 연결된 노드쌍만 추출
- `a = W_att @ concat(...)`: edge별 raw score 생성
- `e = leaky_relu(a)`: 비선형 점수화
- `W_alpha = softmax2D(E, 1)`: 이웃 축 기준 정규화

4 노트북 2부: Cora/CiteSeer 실험

PyG의 Planetoid 데이터셋을 사용해 Cora와 CiteSeer를 불러온다.

```
dataset = Planetoid(root=".", name="Cora")
data = dataset[0]
```

모델은 2층 GATv2Conv로 구성된다.

```
class GAT(torch.nn.Module):
    def __init__(self, dim_in, dim_h, dim_out, heads=8):
        super().__init__()
        self.gat1 = GATv2Conv(dim_in, dim_h, heads=heads)
        self.gat2 = GATv2Conv(dim_h*heads, dim_out, heads=1)
```

Forward에서는 dropout-eLU-dropout-GAT의 전형적 구조를 사용한다.

```
h = F.dropout(x, p=0.6, training=self.training)
h = self.gat1(h, edge_index)
h = F.elu(h)
h = F.dropout(h, p=0.6, training=self.training)
h = self.gat2(h, edge_index)
```

PyG 구현에서 짚고 넘어갈 포인트

- 첫 층 **multi-head**: 여러 head가 서로 다른 방식으로 이웃 중요도를 보게 해 표현력을 높인다.
- 두 번의 **dropout**: 입력 특징과 은닉 표현 모두에 regularization을 걸어 Planetoid 계열 데이터셋에서 과적합을 줄인다.
- 학습 마스크: 손실은 train mask에 있는 노드에 대해서만 계산되므로, 전체 그래프를 보더라도 supervised signal은 제한적이다.

노트북 설명을 발표용으로 정리하면, “attention 메커니즘 자체”보다도 작은 라벨 수와 작은 그래프에서 일반화가 되도록 어떤 학습 장치를 같이 쓰는가가 실험 성능을 좌우한다.

실험 설정

- 데이터셋: Cora, CiteSeer
- optimizer: Adam, learning rate 0.01, weight decay 0.01
- epoch: 100
- dropout: 0.6
- attention heads: 첫 층 8개, 출력층 1개

5 결과 해석

노트북 출력 기준 테스트 정확도는 다음과 같다.

- Cora: **81.10%**
- CiteSeer: **68.10%**

같은 모델이라도 데이터셋 특성에 따라 성능 차이가 크다. 이 차이는 다음 요인의 조합으로 설명할 수 있다.

- 그래프 구조 밀도와 커뮤니티 분리도
- 특징 벡터의 정보량
- train/val/test split의 난이도 차이

결과를 읽는 법

노트북의 해설은 평균 정확도 숫자 하나만 읽지 말라고 강조한다.

- Cora와 CiteSeer의 성능 차이는 모델 차이만이 아니라 데이터셋 성질 차이를 반영한다.
- 따라서 “GAT가 항상 더 좋다”가 아니라 “어떤 구조에서 attention이 더 잘 작동하는가”를 묻는 것이 맞다.
- 교재 그림 7.5–7.6은 성능 숫자보다 해석 포인트와 한계를 읽는 데 초점을 둔다.

6 Degree별 성능 분석

노트북 후반부는 노드 degree 구간별 정확도를 계산한다.

```
degrees = degree(data.edge_index[0]).numpy()

for i in range(0, 6):
    mask = np.where(degrees == i)[0]
    accuracies.append(accuracy(out.argmax(dim=1)[mask], data.y[mask]))

mask = np.where(degrees > 5)[0]
accuracies.append(accuracy(out.argmax(dim=1)[mask], data.y[mask]))
```

이 분석은 평균 정확도 한 줄로 보이지 않는 현상을 보여준다.

- 저차수 노드: 이웃 정보가 부족해 예측이 불안정할 수 있다.
- 고차수 노드: 정보가 풍부하지만 과도한 혼합으로 잡음이 들어올 수도 있다.

왜 이 분석이 중요한가

전체 정확도만 보면 다음 문제가 가려질 수 있다.

- 저차수 노드에서만 성능이 무너지는지
- 고차수 노드에서 과도한 혼합이 생기는지
- 특정 구조 구간이 모델의 취약점인지

즉, degree별 성능은 “모델이 어디에서 실패하는가”를 구조적으로 진단하는 도구다.

7 발제 포인트

발표에서는 아래 질문을 던지면 토론이 좋아진다.

1. attention 계수가 항상 해석 가능한 중요도인가?
2. heads 수를 늘리면 언제 성능이 좋아지고 언제 과적합되는가?
3. heterophily 그래프에서도 동일한 attention 설계가 유효한가?

노트북에서 바로 이어갈 추가 실습

1. heads=4, 8, 16으로 바꿔 성능과 분산 비교
2. dropout=0.3, 0.6, 0.8 비교
3. 같은 split에서 GATv2Conv와 GCNConv 비교
4. 오분류 노드의 degree와 이웃 라벨 분포 분석

8 Chapter 7 핵심 요약

- GAT는 메시지 패싱에 학습형 이웃 가중치(attention)를 도입한다.
- toy 예제는 수식과 구현의 대응관계를 이해하는 데 유용하다.
- PyG 구현에서는 multi-head, dropout, train mask가 함께 작동해 일반화 성능을 만든다.
- Cora/CiteSeer 실험은 데이터셋 성질에 따른 성능 차이를 보여준다.
- degree별 성능 분석은 모델의 구조적 편향을 진단하는 좋은 도구다.